The configuration is based on a file mounted as a volume, similar to how we configured PostgreSQL. Create a docker/rabbitmq folder within your `polar-deployment` repository, and add a new rabbitmq.conf file to configure the default account.

Listing 10.2 Configuring RabbitMQ default account

```
default_user = user
default_pass = password
```

Next, open a Terminal window, navigate to the folder where your docker-compose.yml file is located, and run the following command to start RabbitMQ:

```
$ docker-compose up -d polar-rabbitmq
```

Finally, open a browser window and navigate to http://localhost:15672 to access the RabbitMQ management console. Log in using the credentials we defined in the configuration file (`user/password`) and have a look around. In the following sections you'll be able to follow the message flows between Order Service and Dispatcher Service in the Exchanges and Queues areas of the management console.

When you're done exploring the RabbitMQ management console, you can shut it down as follows:

```
$ docker-compose down
```

Spring Cloud Stream helps integrate applications with event brokers like RabbitMQ seamlessly. But before we get to it, we need to define the logic that will process messages. In the next section, you'll learn about Spring Cloud Function and how to implement the business logic of the new order flow in terms of suppliers, functions, and consumers.